PGR111, solutions week 05

Task 1 - Concepts & Terminology

Briefly explain the following terminology and database concepts:

- Subquery
- VIEW

NB: Keep your answers. They might become helpful later in the semester? :-)

Look at the slides, the textbook or on the internet for answers to this one.

Task 2 – Subquery

You will use the "world" database for the following tasks:

a) *(Hard)* We want to see every country in the world that has no cities. Sort them alphabetically by name. Use a *subquery* for this. (Hint: The countries without cities are the country codes found in country but not found in city. In SQL, we can get the difference (remaining part) between two columns by using “NOT IN”.)

```
SELECT Name
FROM country
WHERE Code NOT IN (SELECT CountryCode FROM city)
ORDER BY Name;
```

Note: We want a subquery for task a), above, but it can also be done with a join. Try both? :-)

```
SELECT country.Name
FROM country
LEFT JOIN city ON Code = CountryCode
WHERE city.Name IS NULL  -- Can use any column from city for this
ORDER BY country.Name ASC;
```

b) *(Hard)* We want to see what percentage the inhabitants of a country represent of Earth's total population. Specifically, we want the country name, the percentage (with column alias "WorldPopPercentage") and surface area for the country. Sort the answer by decreasing percentage. Note: To find the percentage, we first need to calculate the total population, do this as a subquery. (Hint 1: Subqueries can be placed elsewhere than in the where clause. Where do we need a subquery in this case? Hint 2: The SELECT part of a query don't need to display columns as-is: We can combine different values with math, etc.)

```
SELECT Name,
    Population / (SELECT SUM(Population) from country) * 100 as WorldPopPercentage,
    SurfaceArea
FROM country
ORDER BY WorldPopPercentage DESC;
```

Task 3 - VIEW

You will work on the "school" database, see script *DB1102-PGR111_05_schema-and-data-EN.sql*.
Note: The grades are ordered so that 1.0 is the best possible result.

a) Create the following report (query) based on the database tables:
Exam transcript (snr, name, grade) for a specific course on a specific date. For example: cnr = 1 and date: 20.05.2008. (Hint: You need to JOIN two tables.)

```
SELECT student.snr, name, grade
FROM student
JOIN exam ON student.snr = exam.snr
WHERE cnr = 1
AND date = STR_TO_DATE('20-05-2008', '%d-%m-%Y');
```

We have 3 user groups:

1. The student council,
2. The headmaster
3. The study administration.

Views should be created for these user groups.
Remember to test the views with: `SELECT * FROM viewname;`

b) The student council should have access to a view that gives access to the students' names and addresses ("studentcouncil_studentdata_view").

```
CREATE OR REPLACE VIEW studentcouncil_studentdata_view AS
SELECT name, address
FROM student;

SELECT * FROM studentcouncil_studentdata_view;
```

c) The headmaster should have access to a view that shows course number and best grade for each course. (Remember that the best grade is 1.0. The lowest number, in other words.)

```
CREATE OR REPLACE VIEW headmaster_bestgrade_view AS
SELECT cnr, MIN(grade) AS bestgrade
FROM exam
GROUP BY cnr;

SELECT * FROM headmaster_bestgrade_view;
```

d) The student administration should have access to the following views:
   I. studyadm_examoverview_view (*bestgrade, cnr, snr*) <-- Best exam result for each student for each course. Ordered by course number, then by student number.

```
CREATE OR REPLACE VIEW studyadm_examoverview_view AS
SELECT MIN(grade) AS bestgrade, cnr, snr
FROM exam
GROUP BY cnr, snr
ORDER BY cnr, snr;

SELECT * FROM studyadm_examoverview_view;
```

II. *(Hard)* studyadm_studypoints_view (*snr, name, totalstudypoints*)
totalstudypoints = sum of the study points (ECTS) in courses for each student. Remember: Each course only counts its ECTS once, even if you take several exams in it. (Hint: You can use the view from the task above as a JOINable part of this query. And maybe you want to join with another table as well, for 3 tables/views total?)

```
CREATE OR REPLACE VIEW studyadm_studentstudypoints_view AS
SELECT s.snr, s.name, sum(c.studypoints) AS totalstudypoints
FROM student AS s
JOIN studyadm_examoverview_view AS e ON s.snr = e.snr
JOIN course AS c ON e.cnr = c.cnr
GROUP BY s.snr, s.name;

SELECT * FROM studyadm_studentstudypoints_view;
```

Create the following report (query) based on the database tables and the created views:

e) *(Hard)* Transcript of grades (*cnr*, *cname*, *bestgrade* (best grade in course), *studypoints*) for student with snr = 1. (Hint: You can JOIN with the same view as above in this query.)

```
SELECT course.cnr, cname, bestgrade, studypoints
FROM course
JOIN studyadm_examoverview_view ON course.cnr = studyadm_examoverview_view.cnr
WHERE snr = 1;
```

Updates:

f) The Student Council has been tasked with reviewing the information about the students and correcting any errors. They find an error. The student Ola will have the address “QVEIEN 12”. Create an SQL query that they can use to make the change. (*Note:* They only have access to their view: studentcouncil_studentdata_view). Check that the change is made to the underlying table (by using a SELECT statement on it after the update).

```
UPDATE studentcouncil_studentdata_view
SET address = 'QVEIEN 12'
WHERE name = 'OLA';

SELECT * FROM Student WHERE name = 'OLA';
```

g) The headmaster wants to change the best grade in the course with cnr = 1 to 1.0. He knows a little SQL, so he tries the following query towards his view:

```
UPDATE headmaster_bestgrade_view
SET bestgrade = 1.0 WHERE cnr = 1;
```

It doesn’t work! Explain why.

The query gives us this error Code: 1288. “The target table headmaster_bestgrade_view of the UPDATE is not updatable.” The reason is that the View is built up by using GROUP BY. (For more info on restrictions, look up the topic in today’s slides, the textbook, or online.)

Task 4 – Self-study on some extra SQL keywords

You will use the "world" database for the following tasks. ***Note:*** You are meant to read up on the keywords yourself, they have not been explained in class. (Look them up in the textbook or online.)

a) Use a CASE select query to extract: Country name and government form, where values for the latter are shown as: "Monarchy", "Republic" or "Other". Sort the result by country name.

```
SELECT name,
  CASE
    WHEN GovernmentForm LIKE '%Monarchy%' THEN 'Monarchy'
    WHEN GovernmentForm LIKE '%Republic%' THEN 'Republic'
    ELSE 'Other'
  END AS Governance
FROM country;
```

b) Do you remember the JOIN task with every country in the world and their cities? Use COALESCE to clarify that the country has no cities. In other words: All countries in the world and their cities (if they have cities). If the country has no city, the result should state “<< Has no cities >>” (Instead of stating “null”.)

```
SELECT co.Name AS Country, COALESCE(ci.Name, '<< Has no cities >>') AS City
FROM country co
LEFT JOIN city ci ON co.Code = ci.CountryCode
ORDER BY co.Name;
```